

boaz_학기 세션 과제(5)

TabNet

1. Abstract
2. Introduction
 - 2.1. 기존 모델
 - 2.2. TabNet 제안
3. Methods
 - 3.1. TabNet for Tabular learning
 - 3.2. TabNet architecture
 - 3.3. Tabular self-supervised learning
4. Experiments
 - 4.1. **Instance-wise feature selection**
 - 4.2. **Performance on real-world datasets**
 - 4.3. Self-supervised learning
5. Conclusion

TapPFN

1. Abstract
2. Introduction
 - 2.1. 모델 배경
 - 2.2. 모델 제안
3. Methods
 - 3.1. Cell-based Representation
 - 3.2. Two-way Attention Mechanism
 - 3.3. Output
4. Experiments
 - 4.1. 정성적 분석
 - 4.2. 정량적 분석
5. Conclusion

TabNet

1. Abstract

- tabular domain의 딥러닝 아키텍처
- Decision Tree의 feature selection을 Attention으로 구현하여 제작
- 이를 통해 high-performance와 interpretable을 얻고 SSL 방법도 제시

2. Introduction

2.1. 기존 모델

- 딥러닝 → 여러 task에서 representation을 잘 담아내는 방식으로 잘 작동함
 - Tabular(정형화 데이터) → 딥러닝 대신 decision tree의 양상블이 지배적으로 사용됨
 - tabular에서의 hyperplane boundary를 표현하기 효과적
 - decision tree 구조는 내부 조건을 알 수 있어 해석하기에 용이
 - 학습이 빠름
- ⇒ 기존의 DNN 구조가 정형화 데이터에 적합하지 않다고 판단

⇒ 적절한 inductive bias가 없어 optimal solution을 찾기 쉽지 않음

2.2. TabNet 제안

- TabNet은 preprocessing이 필요가 없고 바로 raw tabular data를 gradient descent로 학습
- 각 feature를 고를 때 sequential attention을 사용
 - 이를 통해 interpretability를 얻을 수 있어 학습에 사용되는 능력이 특정 feature에 집중되기 때문에 더 잘 학습함
 - data sample마다 따로 진행되어 각 data마다 다른 feature 선별
- 2가지 해석 가능성
 - 각 sample에 대한 local interpretability: 각 feature이 얼마나 중요한지
 - 전체 dataset에 대한 global interpretability: 전체 dataset에서 각 특징이 얼마나 기여하는지
- unsupervised pre-training을 통해 성능을 끌어냄

3. Methods

3.1. TabNet for Tabular learning

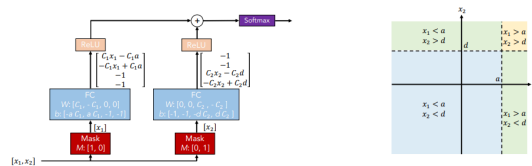
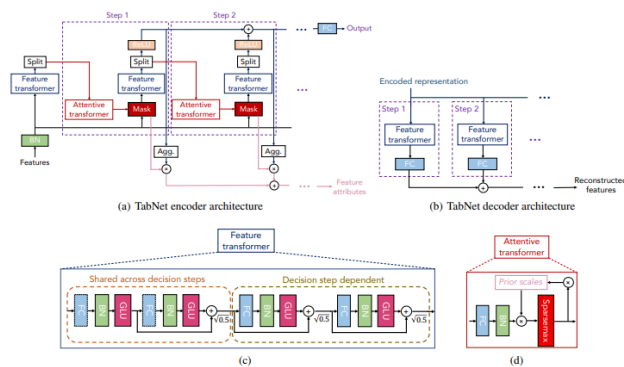


Figure 3: Illustration of DT-like classification using conventional DNN blocks (left) and the corresponding decision manifold (right). Relevant features are selected by using multiplicative sparse masks on inputs. The selected features are linearly transformed, and after a bias addition (to represent boundaries) ReLU performs region selection by zeroing the regions. Aggregation of multiple regions is based on addition. As C_1 and C_2 get larger, the decision boundary gets sharper.

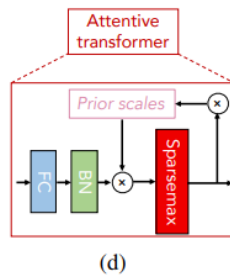
- Mask로 각 input을 독립적으로 받은 후 의도적으로 weight를 조절해서 특정 기준선을 정하고 이를 ReLU에 통과시킴
 - $x_1 > a, x_2 > d$ 이면 0번과 3번의 index만 사용하고 나머지는 0
 - $x_1 < a, x_2 > d$ 이면 1번과 3번의 index만 사용하고 나머지는 0
 - $x_1 > a, x_2 < d$ 이면 0번과 4번의 index만 사용하고 나머지는 0
 - $x_1 < a, x_2 < d$ 이면 1번과 4번의 index만 사용하고 나머지는 0
- ⇒ 이러한 특징 경계로 각 feature의 selection이 key가 됨
- ⇒ 이를 바탕으로 decision tree 성능을 향상

3.2. TabNet architecture

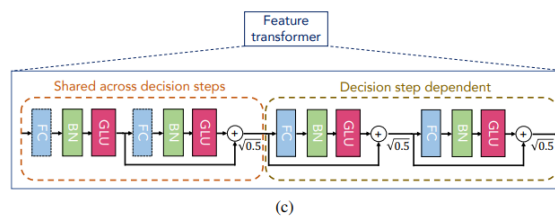


- **Tabnet encoder architecture**

- Feature transformer
 - Attentive transformer
 - Feature masking
 - 매 스텝의 Split 블록에서 이전의 feature transformer에서 나온 representation을 output으로도 보내고 다음 attentive transformer에도 보내 mask를 sequential하게 학습
 - 학습한 mask를 모아 global한 feature importance 구함
- **Attentive transformer**



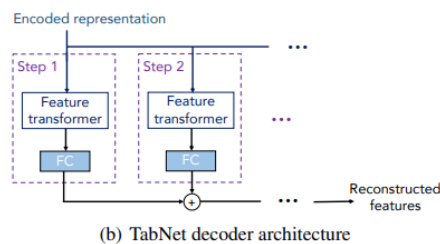
- prior scales - 이전 step에서 각 feature가 얼마 사용되었는지
 - Sparsemax - 구간을 나눠 이번 step에서 중요한 feature은 1 아니면 0으로 mask
- **Feature transformer**



- layer은 각 step이 공유하는 layer와 각 step에 독립적인 layer존재
 - GLU → input으로 들어온 정보를 얼마나 살릴지 결정
- **Mask**
- prior scale term에서 γ 값을 조정하면서 feature selection의 중복 선택 정도를 조절

$$M[i] = \text{sparsemax}(P[i-1] \cdot h(a[i-1])), \text{ where } P[i] \text{ is the prior scale term } P[i] = i \sum_j 1(\gamma - M[j]), w$$

3.3. Tabular self-supervised learning



- Self supervised learning을 위한 decoder의 구조
- encoded된 representation을 reconstruct하는 것
 - pretext task로 missing column의 값을 예측하는 것을 제시

4. Experiments

4.1. Instance-wise feature selection

- 특정 subset이 output을 determine하는 dataset으로 feature selection 실험

Model	Test AUC					
	Syn1	Syn2	Syn3	Syn4	Syn5	Syn6
No selection	.578 ± .004	.789 ± .003	.854 ± .004	.558 ± .021	.662 ± .013	.692 ± .015
Tree	.574 ± .101	.872 ± .003	.899 ± .001	.684 ± .017	.741 ± .004	.771 ± .031
Lasso-regularized	.498 ± .006	.555 ± .061	.886 ± .003	.512 ± .031	.691 ± .024	.727 ± .025
L2X	.498 ± .005	.823 ± .029	.862 ± .009	.678 ± .024	.709 ± .008	.827 ± .017
INVASE	.690 ± .006	.877 ± .003	.902 ± .003	.787 ± .004	.784 ± .005	.877 ± .003
Global	.686 ± .005	.873 ± .003	.900 ± .003	.774 ± .006	.784 ± .005	.858 ± .004
TabNet	.682 ± .005	.892 ± .004	.897 ± .003	.776 ± .017	.789 ± .009	.878 ± .004

- 위 dataset에서 TabNet은 중요한 feature이 무엇인지 걸러냄

4.2. Performance on real-world datasets

Table 2: Performance for Forest Cover Type dataset.

Model	Test accuracy (%)
XGBoost	89.34
LightGBM	89.28
CatBoost	85.14
AutoML Tables	94.95
TabNet	96.99

- 다양한 dataset에서 TabNet이 다른 model을 능가함

4.3. Self-supervised learning

Training dataset size	Test accuracy (%)	
	Supervised	With pre-training
1k	57.47 ± 1.78	61.37 ± 0.88
10k	66.66 ± 0.88	68.06 ± 0.39
100k	72.92 ± 0.21	73.19 ± 0.15

- Self-supervised learning을 진행한 이후 학습한 것이 더 성능이 잘 나옴
- 특히 data의 숫자가 적을수록 성능 향상의 폭이 큼

5. Conclusion

- 하이퍼파라미터 민감성
 - 최적 성능을 내려면 많은 시간과 노력 필요 데이터셋마다 최적값이 달라 범용적 사용이 어려움
- 높은 계산 비용
 - 순차적 어텐션 구조 특성상 학습 시간이 오래 걸림 대규모 배치 크기 필요
- 노이즈에 민감
 - sparsemax로 변수를 강하게 선택하는 구조라 노이즈가 많은 데이터에서 잘못된 변수를 선택할 위험 존재 노이즈 데이터에서 성능 불안정
- 후속연구 - FT- transformer
 - 수치형, 범주형 모든 피처를 동일한 차원의 토큰으로 변환 하여 Transformer로 피처 간 관계 학습

TapPFN

1. Abstract

- 10,000개 샘플을 가진 표 데이터에서 기존의 모든 방법론을 압도적인 성능과 속도 차이로 능가하는 transformer 기반 모델
- 4시간 동안 튜닝된 SOTA 앙상블 모델의 성능을 2.8초만에 뛰어넘음
- 수백만 개의 합성 데이터셋을 통해 '학습하는 알고리즘' 자체를 학습
- 단순 예측을 넘어 데이터 생성, 미세 조정, 밀도 추정, 임베딩 학습 등 다재다능한 파운데이션 모델의 역량을 갖춘

2. Introduction

2.1. 모델 배경

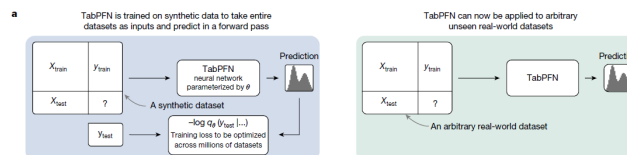
- 정형화된 데이터를 위해 설계된 표 형식의 foundation model
- 표 형식 데이터 예측 분야는 GBDT가 지배, 딥러닝은 이 분야에서 상대적으로 고전
 - 기존 모델들은 데이터셋마다 별도의 학습과 튜닝 과정이 필요하고 지식 전이가 어렵다는 단점 존재

2.2. 모델 제안

- 수백만개의 합성 데이터셋을 통해 학습 알고리즘 자체를 학습함
- 새로운 데이터셋이 들어와도 추가적인 학습이나 하이퍼파라미터 튜닝없이 즉각 예측이 가능

3. Methods

3.1. Cell-based Representation



- 표의 각 셀에 별도의 표현을 할당해 표의 행과 열의 구조를 모델이 직접적으로 이해할 수 있도록 함
- 학습 방식
 - pre-training
 - 다양한 합성 데이터셋을 통해 '어떤 데이터가 주어져도 예측을 잘하는 일반적인 학습 알고리즘' 자체를 트랜스포머에 내재화
 - 과정은 한 번만 수행됨
 - 추론
 - 실제 데이터셋을 받은 훈련 데이터와 테스트 데이터를 입력 받고 순전파(forward pass)로 데이터의 패턴을 파악하고 예측
 - train-state caching
 - 학습 데이터에 대한 연산 결과를 저장해 두었다가 여러 테스트 샘플에 재사용해 CPU에서 최대 800배까지 추론 속도를 높임

3.2. Two-way Attention Mechanism

- 1D Feature Attention
 - 각 셀은 동일한 행(샘플) 내의 다른 피쳐들을 바탕으로 개별 샘플의 특징을 파악
- 1D Sample Attention

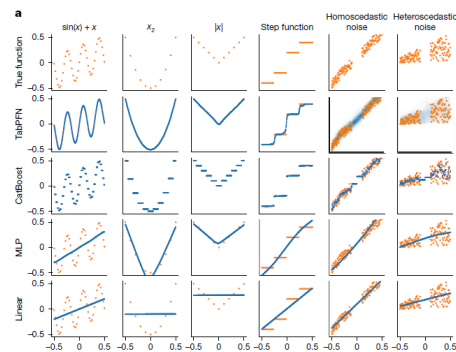
◦ : 각 셀은 동일한 열(피쳐) 내의 다른 모든 샘플들과 소통하여 데이터셋 전체의 통계적 특성과 패턴을 학습
 ⇒ 이 설계를 통해 모델은 샘플의 순서나 피쳐의 순서가 바뀌어도 동일한 결과를 내놓는 invariance를 가짐

3.3. Output

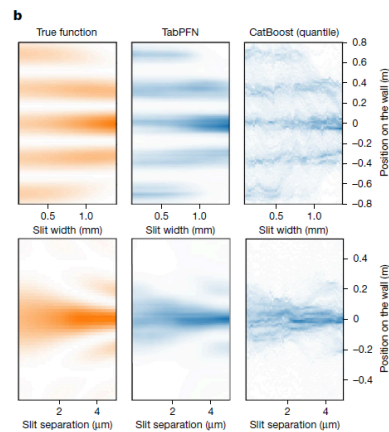
- **분류(Classification):** 소프트맥스(Softmax)를 통해 클래스 확률을 출력
- **회귀(Regression):** 리만 분포를 사용하여 단순한 수치 하나가 아닌 확률 분포 전체를 예측

4. Experiments

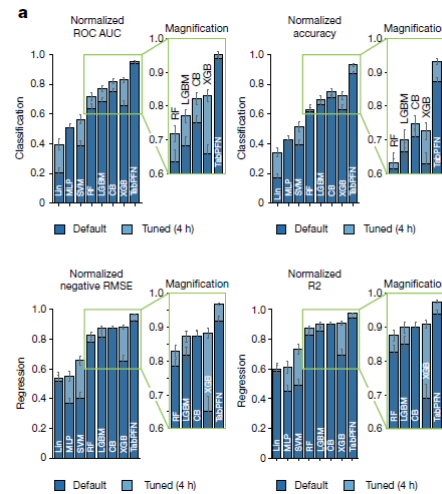
4.1. 정성적 분석



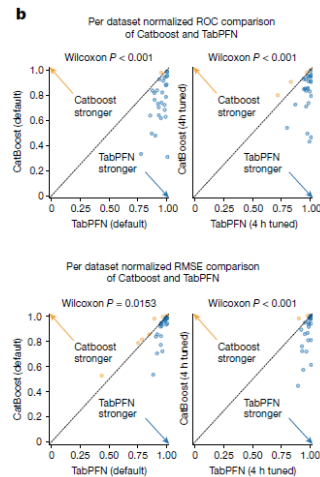
- 대부분의 모델이 단일 예측값을 출력하는 것과 달리 TabPFN은 추가 비용 없이 예측의 불확실성을 포함한 확률 분포를 반환함



4.2. 정량적 분석



- 분류 문제에서 TabPFN이 4시간 동안 하이퍼파라미터 튜닝을 거친 XGBoost, CatBoost 등의 앙상블 모델보다 더 높은 성능을 기록
- 러닝 모델에 일반적으로 취약하다고 알려진 상황에서도 매우 강한 성능을 유지함
- 데이터셋의 특징(결측치 유무, 특징 수 등)에 거의 영향을 받지 않



- AutoML 앙상블 기법과 비교시 AutoGloun의 성능을 뛰어넘음

5. Conclusion

- 1) 밀도 추정: 데이터의 확률 밀도를 추정하여 사기 탐지, 이상 탐지 등에 활용
- 2) 데이터 생성: 실제 데이터의 특성을 모방한 새로운 데이터를 합성하여 데이터 증강 등에 사용
- 3) 임베딩 학습: 다운스트림 작업에 재사용 가능한 의미 있는 특징 표현(representation)을 학습
- 4) 미세 조정 (Fine-tuning): 특정 도메인의 데이터셋에 미세 조정을 통해 성능을 더욱 향상시킴

- 해석 가능성

SHAP과 같은 기법을 통해 각 특징이 예측에 미친 영향을 계산해 모델의 결정을 해석하고 신뢰성을 높임